Abbreviations in mshell

Design draft — 2026-10-06, branch `abbr-config`

Written by Claude from the code on `main` (f0704c8) and a source-level survey of fish 4.9, Nushell 0.116 / reedline 0.52, Elvish 0.21+, zsh-abbr 6.5 and xontrib-abbrevs.

1 Summary

The interactive shell expands a short word into longer text as you type, as fish's `abbr` does. Today the table is hard-coded in `mshell/Main.go`. This design moves it into `init.msh`, typed and checked like `setPrompt`, with the features of fish 4.9's `abbr`:

- a word that expands only in **command position** (the default) or **anywhere**;
- a word that expands only as an **argument of certain commands** (fish's `--command`);
- a word matched by a **regular expression** instead of a fixed word;
- an expansion computed by a **quotation** at the moment it is typed (fish's `--function`);
- a **cursor marker** in the expansion (fish's `--set-cursor`);
- add, erase, rename, list and show.

The user writes:

```
's'  'git status -u' abbr                              # the common case
{abbr: 'co' abbrWord, expansion: 'checkout' abbrText, position: ['git'] abbrArgOf} abbrAdd
{abbr: 'gcm' abbrWord, expansion: "git commit -m '%'" abbrText, cursorMarker: '%'} abbrAdd
{abbr: '\.\.+' abbrRegex, expansion: (upDirs) abbrFunc} abbrAdd
```

The pieces are two built-in record types and three built-in enums: what word is abbreviated, where it expands, and what it expands to; four builtins (`abbrAdd`, `abbrErase`, `abbrRename`, `abbrList`) and two standard library definitions (`abbr`, `abbrShow`). Where fish has a known problem (a trailing space after a cursor marker, no expansion on Enter when the line has a syntax error, quoted words, erasing an abbreviation that is restricted to several commands) the design chooses the other way, and says why.

2 What exists today

`Main.go` declares `var aliases map[string]string` and fills it in `InteractiveMode` with fourteen entries (`s`, `v`, `mk`, `ls`, `gi`, `a`, `d`, `dc`, `c`, `p`, `u`, `gu`, `ga`, `fp`). It is expanded in two places:

- **Space**: the word is the text back to the previous space or `[`. If it is a key, it is replaced by the value and a space.
- **Enter**: only when the cursor is at the end of the line, the same lookup on the last word.

Ctrl-Space inserts a space without expanding. History receives the expanded line.

Problems with it, apart from being hard-coded:

1. **It expands anywhere.** `echo s` followed by Space gives `echo git status -u`. `ls` expands to `eza -l` as an argument too.

2. **It expands inside strings.** `"a s` followed by Space expands the `s`, since only spaces and `[` delimit a word.
3. **Code and commands are mixed.** `u` expands to `'..' cd` and `gu` to `[git add -u]? ([git status -u];) iff`. Those are mshell code, which only makes sense at the start of the line; `s` is a command, which also makes sense at the head of a list (`[s] *`).
4. **The painter does not know about them.** While you type `s` it is shown as a plain word, not a command.
5. The feature is not in the user documentation; `doc/mshell.md` mentions only that Ctrl-Space inserts a space "without expanding aliases".

3 Prior art

3.1 fish 4.9

Fish defines abbreviations with `abbr`, in `config.fish`.

```
abbr --add NAME [--position command | anywhere] [-r | --regex PATTERN] [-c | --command COMMAND]
                [--set-cursor[=MARKER]] [--description DESC] ([-f | --function FUNCTION] |
EXPANSION)
abbr --erase [--command COMMAND]... NAME ...      abbr --rename [--command COMMAND]... OLD NEW
abbr [--show]      abbr --list      abbr --query NAME ...
```

The behavior, read from `src/abbrs.rs`, `src/builtins/abbr.rs` and `src/reader/reader.rs`:

- **Names.** A name is non-empty and has no whitespace. With `--regex`, the name only identifies the abbreviation for list, erase, rename and query.
- **Position.** `command` (the default) is the command word of a statement: the start of the line, after `;`, `|`, `&`, `&&`, `||`, `and`, `or`, `not`, `if`, `while`, inside `(...)`, and after the decorators `command`, `builtin` and `exec` (4.0). It is **not** after `sudo` or `env`; those are ordinary commands, so the next word is an argument. `anywhere` is any token.
- **`--command CMD`** (4.0) expands only in an argument of `CMD`, compared by the command word's raw text. It may be repeated, and it implies `anywhere`. Since 4.2 the same name may exist with different command lists, and erase and rename take `--command` to say which one; an entry added for `git` and `hg` cannot be erased for `hg` alone (open issue #12568).
- **Regex.** PCRE2, wrapped as `^(?:PATTERN)$`, so it must match the whole token.
- **Function.** Called with the token. Exit status 0 replaces the token with the output; any other status leaves it, and the next matching abbreviation is tried. It may read the command line but not change it.
- **Cursor.** `--set-cursor` (marker `%` by default) moves the cursor to the first marker, which is deleted. The trigger character is still inserted, at the end of the expansion; issue #11228 asks for a way to avoid that trailing space.
- **Triggers.** Space, `;`, `|`, `&`, `>`, `<` and `)` insert themselves and then expand the token before the cursor, anywhere in the line. Enter expands the token that contains the cursor. Ctrl-Space inserts a space without expanding, and so does Space-then-Enter, since the cursor is then no longer next to the word. Tab completion and pasted text do not expand.
- **Enter and syntax errors.** Fish checks the line before expanding; a line with a syntax error is not expanded (issue #11710).
- **Tokens and quotes.** Tokens come from the parser, as raw source text, so `'gco'` keeps its quotes and does not equal `gco`.

- **Precedence.** One list in the order of definition, searched from the newest; the last added wins, whether literal or regex. One expansion per trigger.
- **Storage.** In memory for the session, defined in `config.fish`. Before 3.6 each abbreviation was a universal variable; that was dropped because "they have too much metadata to cram into a variable" (PR #9313). The same PR considered, and dropped, keeping the short form in history (`--quiet`) and expanding only on space or only on Enter (`--trigger-on`).
- **History** gets the expanded line. **Undo** (Ctrl-Z) restores the word as one step.
- **Highlighting**: a word in command position that matches an abbreviation is colored as a valid command, even when its function would decline (issue #11809).
- **Completion**: names (not regexes) are offered in command position, described as `Abbreviation: <expansion>`.

3.2 Nushell 0.113+ (reedline)

Since 0.113 (May 2026) `$env.config.abbreviations` is a record of names to strings. Space and Enter expand the whitespace-delimited word before the cursor. Nushell refuses to expand inside strings and in the arguments of external commands, by asking its highlighter what the word is. There is no regex, function, cursor marker, position option, literal-space key, highlighting or completion of them. History gets the expanded line. `abbr list` lists them; there is no command to add or erase one. Before 0.113 the workaround was a menu bound to Ctrl-Space that looked up `alias` definitions, which only worked when the alias was the whole line.

3.3 Elvish

Elvish has three maps from string to string, assigned in `rc.elv`, with no functions or regex (a regex request, #646, was declined):

- `edit:abbr` expands the moment its text has been typed, with no trigger, anywhere: `||` becomes `| less`.
- `edit:small-word-abbr` expands when the next character is of a different kind (space, letter or digit, other). So with `k` for `kubectl`, `ls -k` followed by Space gives `ls -kubectl` (#1472).
- `edit:command-abbr` expands a bare word in command position (start, `|`, `;`, `(`, `{`), found by a regular expression, when followed by whitespace at the end of the buffer. It does not expand on Enter.

Elvish has no undo in its line editor (#636).

3.4 Others

- **zsh-abbr** has "regular" (first word of the line) and "global" (anywhere) abbreviations, configurable **prefixes** that do not count against the start of the line (`sudo ` by default), multi-word names (`git cp`), a cursor marker that drops the trigger key when used, and a "user" scope written to a file that every open session reloads.
- **xontrib-abbrevs** (xonsh) allows a callable `(buffer, word) -> str`, and pressing Space again right after an expansion reverts it.

3.5 Comparison

	fish 4.9	**Nushell**	**Elvish command-abbr**	**This design**
Triggers	Space `; | & > < )`, Enter	Space, Enter	Whitespace at end of buffer	Space `| ; < > ] )`, Enter
Literal space	Ctrl-Space	none	none	Ctrl-Space (exists)

	fish 4.9	**Nushell**	**Elvish command-abbr**	**This design**
Word	parser token, raw text	whitespace word	regex bareword	separator-delimited word, not in a string, path or comment
Quoted word	does not equal the name	no expansion	no expansion	no expansion
Positions	command, anywhere, `--command`	command, not in strings or external args	command	command, anywhere, argument of
Regex	PCRE2, whole token	no	no	Go `regexp`, whole word
Function	function, status and output	no	no	quotation `(AbbrInfo --Maybe[str])`
Cursor marker	first marker; trigger still appended	no	no	first marker; a space trigger is dropped
Precedence	last added; failing function falls through	map key	map key	last added; `none` falls through
Enter on a bad line	no expansion	expands	no expansion on Enter	expands, then checks
Storage	`config.fish`	`$env.config`	`rc.elv`	`init.msh`, checked
Undo	one step	probably one step	no undo	no undo yet (section 6.9)
Highlight, complete	yes, yes	no, no	no, no	yes, yes

4 Principles

These come from the code and from how the prompt was done.

1. **Configured in `init.msh`, checked like the rest of it.** There is one configuration file, and it is fully type checked at startup. A wrong quotation signature or a misspelled position is a type error before the first prompt, not a surprise at a keystroke. This is fish 3.6's choice too, and for fish's reason: an abbreviation now has too much in it for a flat file or a variable.
2. **Enums for choices, records for independent settings.** The abbreviated word is a fixed word **or** a regex, so it is an enum. Where it expands is one of three things, so it is an enum. What it expands to is text **or** a quotation, so it is an enum. The cursor marker and the description are independent of those and of each other, so they are fields of a record. A builtin taking a record follows `HttpRequest`, `ExtractOptions` and `NumFmtOptions`.
3. **Strings are data.** A word inside a string, a path or a comment never expands, whatever the abbreviation. Fish lets a regex see quotes; mshell does not, because a quoted word on an mshell line is always a value, never something you are typing to be expanded.
4. **Command position from mshell's own structure.** mshell has no `;`, `and` or `if` before a command word. A command starts a line, heads a list (`[git ...]`), or follows `|` on a command line. That is the whole definition.
5. **Small surface.** What can be written in mshell from the builtins goes in the standard library.

5 The design

5.1 Types

These are added to `builtinDeclSource` next to the prompt's types, so they are built in, and their names are reserved, as `PromptItem`'s are.

```
enum AbbrMatch =
      abbrWord str       # this word, exactly
    | abbrRegex str      # any word this regex matches in full
end

enum AbbrPosition =
      abbrCommand        # the command word: start of line, head of a list, after | on a command
line
    | abbrAnywhere       # any word
    | abbrArgOf [str]    # a word after one of these commands
end

enum AbbrExpansion =
      abbrText str                          # replace the word with this text
    | abbrFunc (AbbrInfo -- Maybe[str])     # just the replacement, or none to decline
end

type AbbrInfo = {word: str, command: Maybe[str], line: str}

type Abbr = {
    abbr: AbbrMatch,
    expansion: AbbrExpansion,
    position?: AbbrPosition,   # abbrCommand when missing
    cursorMarker?: str,        # put the cursor where this text is in the expansion
    description?: str,         # shown by abbrShow and completion
}
```

`AbbrInfo` is what a quotation is given:

- `word`: the word being expanded, as typed.
- `command`: the command the word is an argument of (section 6.4), or `none` in command position or at the top level of code.
- `line`: the whole line as it is, before the trigger character. The quotation can read it, not change it.

> **Decision. `abbr` is an enum, and there is no name.** In fish, `NAME` does two jobs: it is the word that expands, and it identifies the abbreviation to `--erase`, `--rename`, `--list` and `--query`. With `--regex` it only identifies, and the pattern does the matching. So `name` and `regex` were not exclusive in fish; a regex abbreviation still needed a name that is never typed. Here the field holds what is matched, `abbrWord` or `abbrRegex`, so a regex and a word cannot both be given, and the checker says so. Nothing else is needed to identify an abbreviation: `abbr` and `position` together are its identity (section 5.2), and erase and rename take an `AbbrMatch`. The field is named `abbr` because it is the abbreviation, the short form, paired with `expansion`, the long one.

> **Decision. An enum for the expansion, not a union.** `expansion: str | (AbbrInfo -- Maybe[str])` would let the common case drop `abbrText`. I tried it: a quotation literal that reads a field of its input, such as `(:word? just)`, is refused there with "'get' reads a value whose type is not known here". A

literal whose words wait for their input type, such as `(1 +)` or `(len)`, checks against a union; a field read does not wait. That is the case that matters here: an abbreviation quotation is given a record, `AbbrInfo`, and almost every one starts by reading a field of it (`:word?`). With the union, every such quotation written in place would need `as (AbbrInfo -- Maybe[str])` after it, which checks since the fix in #361 but is longer than `abbrFunc` and must be written out each time. Only a quotation that calls a def, such as `(upDirs)`, would check without it, since the def's signature fixes the input type. As an enum payload, the literal is given its input type, as a def's quotation parameter gives one, so `(:word? just) abbrFunc` checks. The standard library's `abbr` covers the common case, so the enum costs little. If the checker learns to type a quotation literal from a union member, this could be revisited (Section 10).

Decision. Optional fields, not `Maybe`. `position?: AbbrPosition` rather than `position: Maybe[AbbrPosition]`, so a literal writes only what it uses. A builtin is given a new literal, which may lack an optional key. A list of different literals needs `as [Abbr]`: `[{...} {...}] as [Abbr] (abbrAdd) each`. I checked that this passes.

5.2 Builtins

Word	Signature	Does
`abbrAdd`	`(Abbr -- )`	Adds an abbreviation. One with an equal `abbr` and an equal position is replaced, and the new one is the newest.
`abbrErase`	`(AbbrMatch -- )`	Removes every abbreviation whose `abbr` equals this, in every position. One that does not exist is not an error, as with `unsetenv`.
`abbrRename`	`(AbbrMatch AbbrMatch -- )`	Gives every abbreviation whose `abbr` equals the first the second instead. An error if none has the first or any already has the second.
`abbrList`	`( -- new [Abbr])`	Every abbreviation, oldest first. `position` is always present; the other optional fields only when they were given.

`abbrAdd` checks at run time, and fails like any builtin given a bad value:

- an `abbrWord` is non-empty and has no white space (including Unicode white space) or separator (section 6.2), since such a word could never be typed as one word;
- an `abbrRegex` is non-empty and compiles (Go's `regexp` syntax, as `reMatch` uses); it is wrapped as `^(?:REGEX)$`;
- the cursor marker is non-empty (it may be any length; section 6.5);
- an `abbrArgOf` list is non-empty, and each command in it is non-empty with no whitespace.

The builtins work in scripts too, where they only change a table nothing reads; `init.msh` is loaded for scripts, so it must not fail there.

Note. Identity is `abbr` and the position. Fish 4.2 lets `l` be `ls -alh` as a command and `log` after `jj`, which means a name alone no longer picks one abbreviation, and erase and rename need `--command` lists that match exactly (#12568). Here `abbrAdd` replaces only the entry with an equal `abbr` **and** an equal position, so `l` can have both meanings, and `abbrErase` and `abbrRename` act on every entry with that `abbr`. That is simpler, and nobody needs to erase one of two `ls` from the REPL; editing `init.msh` does that. Two `abbr` values are equal when they are the same member with the same string, as enum values compare: `'l' abbrWord` and `'l' abbrRegex` are different.

5.3 Standard library

```
def abbr (str str -- )
    word!, expansion!
    {abbr: @word abbrWord, expansion: @expansion abbrText} abbrAdd
end

def abbrShow ( -- )
    # One line per abbreviation: the word or regex, where it expands, and the text or <function>.
    ...
end
```

`abbr` is fish's `abbr NAME EXPANSION`: a word in command position, expanded to text. `abbrShow` is fish's `abbr --show`, for reading at the REPL. It does not print source to paste back, as fish's does; that existed in fish to move abbreviations out of universal variables, and mshell has none. Fish's `--list` is `abbrList (:abbr?) map`, and `--query` is `abbrList (:abbr? 'gco' abbrWord =) any`; neither needs a word of its own. I checked the `--query` form against a prototype.

5.4 Examples

```
's'  'git status -u' abbr
'gu' '[git add -u]? ([git status -u];) iff' abbr
'u'  "'..' cd" abbr

# Only as an argument of git: `git co` becomes `git checkout`.
{abbr: 'co' abbrWord, expansion: 'checkout' abbrText, position: ['git'] abbrArgOf} abbrAdd

# The cursor goes between the quotes, with no space after them.
{abbr: 'gcm' abbrWord, expansion: "git commit -m '%'" abbrText, cursorMarker: '%'} abbrAdd

# `..` is `'..' cd`, `...` is `'../..' cd`, and so on.
def upDirs (AbbrInfo -- Maybe[str])
    :word? len 1 - n!
    '..' @n repeat '/' join up!
    $"'{@up}' cd" just
end
{abbr: '\.\.+' abbrRegex, expansion: (upDirs) abbrFunc} abbrAdd
```

I checked `upDirs` against a prototype of these types: it type checks, and gives `'../..' cd` for `...`.

Fish's usual example of a function, `!!` for the last command, has no counterpart here. It reads the history, and an mshell history line is not always a command, so no `history` builtin is added for it.

6 Editing behavior

6.1 When expansion happens

An abbreviation is tried when one of these keys is pressed in the main command line:

- **Space**, and the **separators** `|`, `;`, `<`, `>`, `]`, `)`: the word that ends at the cursor is tried, then the key is inserted. This works anywhere in the line, not only at the end.
- **Enter**: the word that contains the cursor, or ends at it, is tried, then the line runs. The expansion happens **before** the line is parsed and checked, so a line that would not parse still expands (unlike fish, #11710); what runs, and what goes into history, is the expanded line.

Nothing is tried for Ctrl-Space (a literal space, as today), Tab completion, history search, Alt-. or pasted text. To run an abbreviated word as it is, type Ctrl-Space and then Enter: the cursor is then no longer next to the word.

`]` and `)` are triggers because a list head is often closed straight away: `[s]` should become `[git status -u].` `;` stays for fish parity; `[s];` is already handled by `]`.

6.2 The word

The **separators** are white space, as the lexer defines it (Go's `unicode.IsSpace`, so a no-break space counts), and `[ ] ( ) { } | ; < >`. Using the lexer's definition keeps the word the same as what the line means when it runs. The word is the text between the nearest separator (or the start of the buffer) and the cursor; on Enter, it extends right to the next separator. It is not lexer tokens, since `!!` lexes as two `BANG` tokens and `..` as a `PREFIXQUOTE`, and both are useful abbreviated words.

The word is **not tried** when it is empty, or when the line's lexer (allowing unfinished strings, as the painter does) puts its first character inside a string, format string, path or comment, finished or not. So `"a` `s` followed by Space, `` `s `` and `# s` never expand.

The word is also **not tried** unless both of its ends are grapheme boundaries of the line. The cursor is always on one, since the editor moves by grapheme. The other end may not be: a combining mark typed after `[` belongs to the bracket's grapheme, while the word scan, like the lexer, splits by code point and would start the word at the mark. Replacing that word would delete the `[` too, because `replaceText` widens a replacement to whole graphemes. Such a word is left alone. It only arises with a separator followed by a combining mark, a zero width joiner or a spacing mark, or (on Enter) a prepended mark before a separator.

6.3 Matching and precedence

An abbreviation **matches** a word when its `abbr` is `abbrWord w` and `w` equals the word, or `abbrRegex r` and `r` matches the whole word.

Equal means the same UTF-8 bytes, which is the same sequence of code points: there is no Unicode normalization and no case folding. That is how `=`, `match` and definition names compare strings in mshell, and how fish compares abbreviation names. So `é` written as one code point and as `e` with a combining accent are different words; keyboards and editors produce the single code point, so in practice they agree. A regex works on code points, as `reMatch` does: `.` is one code point, not one grapheme. It **applies** when it matches and its position allows the word's position (section 6.4).

The applying abbreviations are tried from the newest to the oldest:

- `abbrText s`: the word is replaced with `s`. Done.
- `abbrFunc q`: `q` is run on an `AbbrInfo`. `just s` replaces the word with `s`, and that is all. `none` declines, and the next applying abbreviation is tried, as in fish. If `q` fails, takes too long or is interrupted, the word is left as it is, the key is inserted, and nothing else is tried (section 6.6).

At most one word is expanded per key, and the expansion is not expanded again.

6.4 Positions

The position of a word comes from the text before it, lexed as the painter lexes it:

- **Command position**, which `abbrCommand` allows:
  1. nothing but whitespace is before the word;

2. the token just before the word is `[`, so the word heads a list: `[s`, `[[s`;
3. the line is a command line (its first word is a program, by the rule the shell uses when it runs the line) and the token just before the word is `|`.

A newline is not on the list: it means nothing in the grammar, so the first word after one is in command position only if one of these holds.

- **An argument of `c`**, which `abbrArgOf [... c ...]` allows, when the word is not in command position and its command is `c`:
  1. inside a list, the command is the first element of the innermost unclosed `[`, if that is a plain word;
  2. otherwise, on a command line, it is the first word of the current part of the pipeline.

  The command is compared as written, so `/usr/bin/git co` does not match `['git']` (as in fish).
- `abbrAnywhere` allows every word.

`AbbrInfo.command` is the word's command as defined above, or `none`. There is nothing after `sudo`, as in fish: in `sudo s`, `s` is an argument of `sudo`. An `abbrArgOf ['sudo' 'doas']` copy of the abbreviation covers it. zsh-abbr's prefixes that do not count as the command are not adopted.

> **Note. `u` and `gu` are mshell code, not commands.** Under `abbrCommand` they also expand at the head of a list, where `['..' cd` makes no sense. Nobody types `[u`, so there is no position for "start of line only". If it becomes a problem, an `abbrLineStart` position can be added without breaking anything.

6.5 The cursor marker

`cursorMarker` is a string of any length, not one character: `'%'`, `'%%'` or `'<cursor>'` are all markers. Any length matters because a one-character marker can collide with the expansion's own text: with `'%'` as the marker, the expansion `date +%Y-%m-%d %` would lose the `%` of `%Y` and put the cursor after `date +`, so that abbreviation would use `'<cursor>'` instead. Fish's `--set-cursor=MARKER` is a string of any length too; its default is `%`, and here there is no default.

When `cursorMarker` is given and the expansion contains it, the first occurrence of the whole marker is removed and the cursor is put where it began. For a function, this applies to the text it returns. Then:

- a **Space** trigger is dropped: the cursor is inside the expansion, and a space at its end is what fish users complain about (#11228). `gcm` and Space give `git commit -m '|'`, with the cursor at `|`.
- any other trigger is inserted at the end of the expansion, since it is syntax the user typed: `[gcm]` gives `[git commit -m '|']`.
- on **Enter**, the marker is removed and the line runs.

The expansion is searched for the marker's bytes. Since both are UTF-8, a match is always a whole sequence of code points, never part of one. The marker need not be whole graphemes. If the cursor would fall inside a grapheme (for example, the marker was followed by a combining mark), it moves forward to the end of that grapheme, as it does after any replacement.

When the expansion does not contain the marker, `cursorMarker` changes nothing. This differs from fish, which keeps the trigger; zsh-abbr and xonsh drop it.

6.6 Running a quotation

A quotation is run as the prompt quotation is, by the same code in `Prompt.go`:

- on its own stack, so it cannot change the values on yours;

- stopped after 1 second, or by Ctrl-C, together with any command it runs. This is its own limit, shorter than the prompt's 3 seconds, since the user is waiting for a key to appear;
- when it fails or is stopped, the word stays, the key is inserted, and the error is shown below the command line, where completion messages go. The same error is not shown again while it keeps failing the same way.

A quotation can run commands, such as `git branch`, but it runs while the user waits on a keystroke, so it should be quick.

6.7 Painting

A word in command position that has an abbreviation applying there (by `abbrWord` or `abbrRegex`) is painted as a command, as a program on the path is (underlined blue). A function abbreviation is not run to decide this, so a word whose function would decline is still painted as a command (as in fish, #11809).

6.8 Completion

Tab in command position offers the `abbrWord` words of abbreviations that apply in command position, next to programs and definitions. `abbrArgOf` abbreviations are not offered for their commands' arguments, nor `abbrAnywhere` ones elsewhere; fish tried the latter in 4.8 and took it back. When completion can show descriptions, `description` (or the expansion's text) is the one to show.

6.9 Undo

The line editor has no undo. When it gets one, an expansion is one step back to the typed word and the key. Until then Ctrl-Space is the way to avoid an expansion, and Backspace over the expanded text the way out of one. There is no special key to revert an expansion, such as xonsh's second Space.

6.10 History

History gets the expanded line, as now, and as in every shell surveyed. Fish's dropped `--quiet` (keep the short form in history) is not adopted: history is the record of what ran.

7 Fish parity

fish	mshell
`abbr NAME EXP`, `abbr --add`	`'NAME' 'EXP' abbr`, or `{abbr: 'NAME' abbrWord, ...} abbrAdd`
`--position command`	`abbrCommand`, the default
`--position anywhere`	`abbrAnywhere`
`--command CMD...`	`['CMD' ...] abbrArgOf`
`--add NAME --regex P`	`abbr: 'P' abbrRegex`, with no name; Go syntax instead of PCRE2
`--function F`	`(F) abbrFunc`; `none` for a non-zero status
`--set-cursor[=M]`	`cursorMarker: 'M'`, a string of any length; there is no default marker, and a space trigger is dropped
`--description D`	`description: 'D'`
`--erase NAME`	`'NAME' abbrWord abbrErase`; erases every position, and a missing one is not an error
`--rename OLD NEW`	`'OLD' abbrWord 'NEW' abbrWord abbrRename`
`--list`	`abbrList (:abbr?) map`
`--query NAME`	a test on `abbrList`
`--show`	`abbrShow`, readable rather than re-runnable

fish	mshell
Space `;` `|` `&` `>` `<` `)` triggers	Space `|` `;` `<` `>` `]` `)`; there is no `&` operator in mshell's command lines
Enter expands the word at the cursor	The same, before the line is checked
Ctrl-Space	The same, as today
Undo is one step	Not yet; no undo in the line editor
Highlight as a command	The same
Complete names in command position	The same, for `abbrWord` words
Universal variables (3.0–3.5)	Not adopted

8 Moving the current table

Delete `aliases` from `Main.go` and put this in `init.msh`:

```
's'  'git status -u' abbr
'v'  'nvim' abbr
'mk' 'mkdir' abbr
'ls' 'eza -l' abbr
'gi' 'nvim .gitignore' abbr
'a'  'git add' abbr
'd'  'git diff -w' abbr
'dc' 'git diff -w --cached' abbr
'c'  'git commit' abbr
'p'  'git push' abbr
'u'  "'..' cd" abbr
'gu' '[git add -u]? ([git status -u];) iff' abbr
'ga' '[git add -A]? ([git status -u];) iff' abbr
'fp' 'git fetch --prune' abbr
```

What changes for these: they expand only in command position, not after `echo`; never in a string; also on `]`, `|`, `;`, `<`, `>`, `)`; and on Enter with the cursor anywhere in the word. The shell ships with no abbreviations (fish issue #9462 asks for defaults; that is a separate question).

9 Implementation plan

1. **Types.** Add the declarations to the built-in declarations (`builtinDeclSource` in `Prompt.go`, or a new `Abbr.go` whose source is joined to it).
2. **State.** `EvalState` gets an ordered list of abbreviations with their compiled regexes, next to `PromptQuote`.
3. **Builtins.** `abbrAdd`, `abbrErase`, `abbrRename` and `abbrList` in `Evaluator.go`, in `BuiltInList.go`, and their signatures in `TypeCoreBuiltins.go`, which must match what the runtime accepts exactly.
4. **Standard library.** `abbr` and `abbrShow` in `lib/std.msh`.
5. **Finding the word.** A function in `Abbr.go` that takes the line, the cursor and a test for "is a program", and returns the word's range, whether it may be tried, its position and its command. It has no terminal state, so it is tested with a table of lines and cursors.
6. **Expanding.** One function that the Space, separator and Enter handlers call, replacing the two copies in `Main.go`. It runs quotations through the prompt's runner (`evalCancel`, killing commands it runs), with its own 1 second limit beside `promptTimeout`.
7. **Painting and completion** in `CommandPaint.go` and the completion code.
8. **Tests.**

- Go: the word and position table (start, list heads, nested lists, `|` on command and code lines, strings, paths, comments, `!!`, `..`, mid-line cursor, Enter in the middle of a word, a no-break space, a combining mark after `[`, an `abbrWord` with an accent); precedence and `none` falling through; the cursor marker with each trigger.
- `tests/success`: `abbrAdd`, `abbrList`, `abbrRename` and `abbrErase` round trips.
- `tests/typecheck_fail`: a quotation of the wrong type, text where `AbbrExpansion` or `AbbrMatch` is expected, redeclaring a built-in name.

9. **Documentation.** An "Abbreviations" section in `getting-started.inc.html` after "Customizing the Prompt"; the builtins in `functions.inc.html`; the built-in type names in `type_system.inc.html` and one line in `mshell.md` (only that the names are reserved, as for the prompt); "without expanding aliases" in `mshell.md` becomes "abbreviations"; an entry in `CHANGELOG.md`.

10 Open question

A quotation literal is not given its input type by a union. The smallest example:

```
def qq (int | ({a: int} -- int) -- ) drop end
(:a?) qq
```

gives "type error at line 2, column 3: 'get' reads a value whose type is not known here (_); annotate it". It checks when the parameter is a plain quotation type, `def qq (({a: int} -- int) -- ) drop end`, and when the body's words wait for their input type, as `(len)` against `int | (str -- int)` does.

The related `as` case, `(:a?) as ({a: int} -- int) drop`, was fixed in #361 (`0af057e`). I checked against a build of that commit: the `as` case checks, and so does `(:a?) as ({a: int} -- int) qq` against the union parameter, so `as` is now a workaround. The union case above still fails. Should a union with exactly one quotation member give a quotation literal that member's input type, as a plain quotation parameter does? The design uses an enum either way.

11 Sources

- fish: abbr docs interactive docs source `src/abbrs.rs`, `src/builtins/abbr.rs`, `src/reader/reader.rs` at master `da63cfd2`; PR #9313 issues #11228, #11710, #11809, #11944, #12568, #9462.
- Nushell: line editor book 0.113 release notes PR #18197 reedline `src/engine.rs`.
- Elvish: edit module reference `pkg/cli/tk/codearea.go`; issues #646, #1472, #636.
- zsh-abbr: zsh-abbr.olets.dev. xonsh: xontrib-abbrevs.